Project: Productionizing your LLM Server

Due: 02/14/2026 11:59 PM
Extended Deadline: 02/15/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Refactor to FastAPI
- Task Overview
- Task

Testing and pre-commit hooks
- Task Overview
- Task

CI pipeline for FastAPI
- Task Overview
- Task

Wrap Up
- Conclusion

Refactor to FastAPI (Task)

Building a Better LLMService

The Flask application we built earlier performs well, but now you want to focus on benchmarking and developing an improved version of the service. You understand that WSGI frameworks support only one request at a time, and you want to utilize your resources more efficiently.

To achieve this, you need an ASGI framework, and you have learned that FastAPI is one of the most modern and popular web frameworks with ASGI capabilities.

In this task, we will build, deploy, and load test a rewritten version of the llmapp. Our primary goal is speed, so we will focus on maximizing the performance of our service without changing its functionality. This represents a specific type of refactor where we preserve the interface while leveraging better tooling and improved systems to enhance overall efficiency.

Before we begin

We will be using the same repository from the earlier project. Ensure that you have accepted the GitHub Classroom invite. If you already have a cloned repository, we encourage you to **remove it and clone the repository again**. This ensures that no additional files beyond the repository code are present in the folder.

Setup: Getting Poetic about Environments

In the previous project, we created and used a Python virtual environment. Virtual environments are the first step in creating isolated Python environments and managing dependencies. As you may remember, we manually updated our `pyproject.toml` file and added dependencies to it in

the previous project. Now, we will explore a modern tool called `Poetry`.

Poetry is a tool for dependency management and packaging in Python. Poetry requires you to declare dependencies so it can resolve compatible versions, lock them for reproducible installs, and build correct package metadata. This explicit declaration allows its resolver to compute a consistent environment and produce a lock file that guarantees repeatable builds across machines and over time.

To get started with Poetry, we will use the official installation script provided by Poetry. Make sure to deactivate any virtual environment you currently have activated.

Use the following command:

```
curl -sSL https://install.python-poetry.org |
```

The above command will create a new virtual environment and install Poetry globally. Next, you can add the `PATH` to your environment.

To add the path, run:

```
export PATH="$HOME/.local/bin:$PATH"
```

Additionally, edit your `~/.profile` (or `~/.bashrc` or any other shell configuration file) to add the `PATH` to the Poetry bin. For example, run:

```
vi ~/.profile
```

On the last line of the file, add:

```
PATH="$HOME/.local/bin:$PATH"
```

Make sure that `poetry --version` works and returns a version for Poetry.

For our local development, we are using the installer provided by Poetry. However, for installing Poetry in Docker images and on CI, we will be using `pipx`. Pipx helps install Python CLI tools globally.

Now, let's rename our `flask-app` folder to `llm_server` to keep things cleaner. Note that if you rename it using standard Linux commands, Git will consider this as deleting the `flask-app` folder and adding a new `llm_server` folder. To ensure Git recognizes your intent to rename, use:

```
git mv flask-app llm_server
```

Now, navigate into the `llm_server` directory and delete the existing `pyproject.toml` file, as we will be asking poetry to create one for us.

Once deleted, run the following:

```
poetry init
```

When prompted, keep the package name as `llm_server` and say **no** to interactively installing main or dev dependencies. Complete the setup and you should see a new `pyproject.toml` file.

By default, Poetry creates virtual environments in `{cache-dir}/virtualenvs` instead of your project folder. Here, we would like to keep the virtual environment in the project folder so that deleting the folder also removes the virtual environment.

Create a new file named `poetry.toml` and add the following to it:

```
[virtualenvs]
in-project = true
path = ".venv"
```

Now, let's install everything we need for FastAPI using Poetry:

```
poetry add fastapi
poetry add uvicorn[standard]
poetry add gunicorn
poetry add llama-cpp-python
```

You should observe three important things here:

1. When using Poetry, you don't need to activate or deactivate any environment. This is because Poetry is not a replacement for virtualenv; it creates and uses virtualenv for project isolation.
2. Poetry creates a `.venv` virtual environment in the project folder. This hosts all our dependencies. You shouldn't activate it manually - just use Poetry to interact with it.
3. You must notice that there's also a `poetry.lock` file. This is a very important aspect of using Poetry, as it

allows you to freeze dependencies instead of just stating them.

When managing dependencies, tools like requirements.txt or pyproject.toml let you list required libraries but don't guarantee specific versions. For reproducibility, every dependency - including transitive ones - must have an exact version. Doing this manually is tedious, so modern dependency managers generate lock files that record precise versions for all packages.

You should commit these lock files to Git so others can run `poetry install` and recreate the exact environment you used.

Subtask 1: A Simple Refactor

Your first task is to rewrite your Flask application into FastAPI.

Convert the Flask code written in **llm_server/main.py** into FastAPI code with the same functionality.

Here are some hints on what to change:

- Import the correct FastAPI libraries and remove the Flask ones.
- Declare your FastAPI app.
- Change the routes to use FastAPI's decorators instead of Flask's decorators - see here for a useful example.
- Pay attention to how query parameters are passed.
- Make sure your `tinymodel` file is present in the top-level project directory. Download it again if required.
- Remove the main function and `__name__ == "__main__"` code blocks.

Once you complete the above steps, you should be able to run the application with:

```
poetry run uvicorn llm_server.main:app --relc
```

The `poetry run` syntax allows you to run Python commands using the virtual environment without activating it.

Here, we are using a production web server, `Uvicorn`, with our FastAPI application.

Interim Validation

While your server is running, you can curl into the server and use:

```
curl http://localhost:8000/api?message=hi
curl http://localhost:8000/healthcheck
```

to check that our web service is working as expected.

Before proceeding, make sure that this works.

If everything works, commit and push your changes to GitHub. It's best practice to commit frequently so you can easily revert if needed.

Subtask 2: Being Pydantic for Our Input

In this subtask, we will introduce you to writing "idiomatic" Python code. This involves writing code that adheres to the conventions, best practices, and philosophical principles of the Python language. It emphasizes readability, clarity, and conciseness, making the code easier to understand and maintain for other Python developers.

The effort we invest in refactoring the code to make it more Pythonic (another term for idiomatic Python) pays off by significantly increasing the maintainability of our code.

Starting from Python 3.5, Python provides a mechanism to add type hints to variables so that developers can specify the types for variables. Python, being a dynamically typed programming language, is not responsible for enforcing these types. This gap is filled by data-validation libraries like Pydantic.

Using Pydantic, we create classes that we refer to as **data models**. An example of a Pydantic data model is:

```
from typing import Optional
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str = 'John Doe'
    signup_ts: Optional[int] = None
    friends: list[int] = []
```

One of the core features of FastAPI is that it supports and encourages the use of Pydantic models.

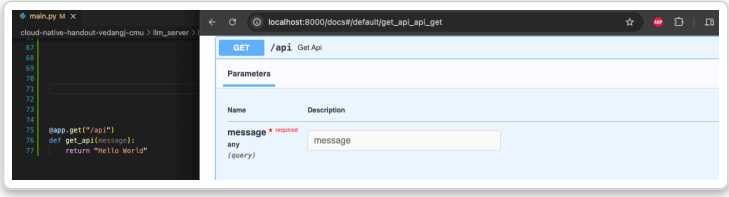
Our first concern with our application will be to add input and output validation to our service. We shall do so with Pydantic validation. These tell FastAPI how it should read input data.

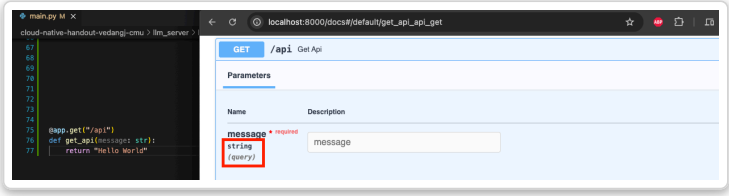
Before you start, make sure you have viewed FastAPI's auto-generated docs page. Access it by navigating to:

```
http://localhost:8000/docs
```

This is a super popular feature that FastAPI provides. You can play with your API—send inputs, see outputs, and make sure the API works.

When you add type hints and Pydantic models to the API endpoint functions, you will see the docs page also reflecting that, and FastAPI automatically rejecting invalid inputs. Observe the difference in the two images.





While this is excellent, it is usually not sufficient for all validation logic. In some cases, you may need to create a custom validator. This is especially important for inputs like strings, where assumptions about the data often extend beyond simple type checks, such as length constraints.

For this class, we shall focus on limiting the number of tokens that our LLM server accepts as input. Here, a token is a portion of a word that a large language model interprets as a single item. When you initialize your Llama object, you specify `n_ctx` to be 512. Currently, if you provide a sufficiently large string:

Bad string:

```
messagemessagemessagemessagemessagmessagemess
```

Your server will error out, and an error will appear in the terminal where you are running the server. FastAPI is intelligent enough to provide you with the error message without breaking your current process. However, we want to provide a more readable error message to anyone who is less familiar with an LLM.

To provide this custom functionality, you must define a custom Pydantic type. You do this by subclassing the

`BaseModel` class from Pydantic, as shown in the following example:

```
from pydantic import BaseModel, field_validat
from fastapi import Query
from fastapi.exceptions import RequestValidat

class MyCoolType(BaseModel):
    cool_item: int = Query("cool_item")

    @field_validator("cool_item")
    def check_cool_item(cls, v: int) -> int:
        if v > 100:
            raise RequestValidationError(f"Yc
        return v
```

This is a particularly complex example, so let's examine it line by line:

1. We import the `BaseModel` and `field_validator` decorator from Pydantic.
2. We create a member called `cool_item`, which is an integer. The `=` sign indicates a default value for this integer, which uses the `Query` class from FastAPI to retrieve the "cool_item" query parameter.
3. We have a decorator with `cool_item` in it that will validate `cool_item`. This decorator enables Pydantic to use this function to check the element before creating the object.
4. The function itself is a class method, so it accepts the class as `cls` and the actual item to validate as `v`. The `->` symbol denotes the return type—what the function expects to return. For a validation function, we take in the class and a value to validate, and return that value if it passes; otherwise, we throw an error.
5. If `cool_item` is greater than 100, we raise a `RequestValidationError`, which informs someone (the cluster owner or auditor) that the request attempted to pass but ultimately failed.
6. If `v` passes this check, we return it.

Your task is to edit the above code to verify that the number of tokens passed to the model is less than or equal to 512, and return an error otherwise.

The name of your validator class should be `LLMPrompt`
The name of the class member should be `message`

If the number of tokens exceeds 512, an error **MUST** be returned containing the message: `"Token count exceeds the maximum allowed limit of 512."`

To get the number of tokens for a string using the `llm` variable, simply use:

```
len(llm.tokenize(v.encode('utf-8')))
```

All this does is:

1. Take `v` and convert it from a string to a list of bytes. You do not need to understand the syntax, only that this is required.
2. Run the tokenizer and retrieve the list of tokens from the model.
3. Get the number of tokens with `len`.

Interim Validation

To use your validator class, you will need to replace the type definition `str` in your input parameter with `LLMPrompt = Depends(LLMPrompt)`. This is simply a way to add custom validators to FastAPI—there is much more to it, but it is not required for this course.

Once replaced, you can check the validation by sending the bad string shown earlier as input from the docs page. You should receive a 422 status code as the response.

While this is substantial work, you can see that it is worthwhile to provide these error messages to consumers of your API.

Now that you've got the validation working, it's a good time to commit and push your changes. Need I remind you the benefits of doing this?

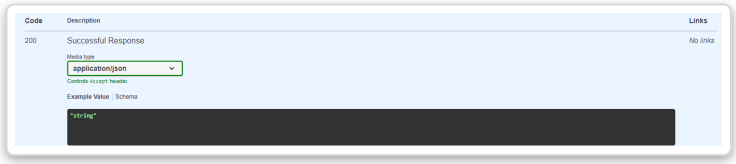
Subtask 3: Pydantic Output Validation

While our input is properly validated now, we can see that our response has no typing.

What we want to do is provide a type for our output, which will allow us to populate the API documentation. The clients of our API can know the output types of our APIs even without running the APIs. If your team is responsible for the

API and another team plans to build a API client for it, both the team can agree on an API documentation even before starting to work on their piece.

Currently, our output looks like the following:



We would like to

Validate our response output
Provide the output model in the docs

To accomplish this, you will need to create a Pydantic type for our output. As a reminder, our output looks like the following:

```
{
  "id": "cmpl-186c31c2-9803-4421-a9aa-81db9ab
  "object": "text_completion",
  "created": 1708482398,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.ggu
  "choices": [
    {
      "text": "\n     A Taco is a dish consis
      "index": 0,
      "logprobs": null,
      "finish_reason": "length"
    }
  ],
  "usage": {
    "prompt_tokens": 41,
    "completion_tokens": 128,
    "total_tokens": 169
  }
}
```

This sounds quite challenging, but we will guide you through it using a top-down approach:

Information

If you want to approach this differently, feel free to do so—this is just one way to solve this task.

To implement the top-down approach, we want to write a single type for each field in our output:

- `id`
- `object`
- `created`
- `model`
- `choices`
- `usage`

Let's start by adding the `BaseModel` subclass:

```
from typing import Optional, List, Any, Typed


class BaseLlamaResponse(BaseModel):
    pass # Delete me when you add the first t
    #TODO
```

To get the API documentation to update with the response type, add `BaseLlamaResponse` to your `@app.get` decorator for the `/api` route:

```
@app.get('/api', response_model=BaseLlamaResp
```

We suggest keeping the docs page open while you work on this—it is much easier to write this iteratively than to attempt to get everything working all at once.

Unlike the model input, where we only cared about the type of the input, for the output we want to provide not just the type but also a default value that helps us understand what is likely to go there.

Let's start with `id`.

For `id`, we know from the above output that it is a string. Let's assign it the `str` type and give it a default value of `"cmpl-186c31c2-9803-4421-a9aa-81db9ab4ea30"`. Your turn now, add the following line to the `BaseLlamaResponse` class:

```
id: str = "cmpl-7fc1be4c-8f5b-4b2f-805f-f8c50
```

You can add the next four fields in the same way. Use the example to add:

- `object`, with type `str` and value `"text_completion"`
- `created`, with type `int` and value `1708459650`
- `model`, with type `str` and value `"tinyllama-1.1b-chat-v1.0.Q2_K.gguf"`

At this point, your output value in the docs should look like:

```
{
  "id": "cmpl-7fc1be4c-8f5b-4b2f-805f-f8c5086
  "object": "text_completion",
  "created": 1708459650,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.ggu
}
```

With the easier parts complete, let's focus on `usage`.

`usage` may seem difficult to write as it is a dictionary. Currently, we haven't provided a way to write a `Dict` type. If you attempt to use `dict`, you will encounter a validation error.

To define a `Dict` as a type, you need to create a separate `BaseModel` and then reference that model within the `BaseLlamaResponse` output.

For example, if you had something like this:

```
{
  "cool": {
    "a": 1,
    "b": "hi",
    "c": 0
  }
}
```

You can do the following:

```
class Outer(BaseModel):
    cool: Inner

class Inner(BaseModel):
    a: int = 1
    b: str = "hi"
    c: int = 0
```

For `usage`, use the example above to write the type. As a reminder, `usage` is a dictionary with three keys:

- `prompt_tokens` is an integer with default value `198`.
- `completion_tokens` is an integer with default value `10`.
- `total_tokens` is an integer with default value `208`.

Interim Validation

Check that your API documentation page example output looks like:

```
{
  "id": "cmpl-7fc1be4c-8f5b-4b2f-805f-f8c5086
  "object": "text_completion",
  "created": 1708459650,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.ggu
  "usage": {
    "prompt_tokens": 198,
    "completion_tokens": 10,
    "total_tokens": 208
  }
}
```

With `usage` complete, we just need to address `choices`. We know that it is a list of dictionaries that we can now type by creating another subclass of `BaseModel`.

You need to import `List` from the `typing` module and use it as:

```
from typing import List

choices: List[MyCoolSubclass]
```

Replace `MyCoolSubclass` with the name of your choices class.

The sub-types and defaults are:

- `text` has type `str` and value `"Beep boop"`
- `index` has type `int` and value `0`
- `logprobs` has type `str | None` and default value `None`
- `finish_reason` has type `str` and value `"stop"`

The `str | None` type means the value of this variable could either be `None` or a `str`.

Validation

If you have completed this subtask as expected, your default output schema value should match this completely:

```
{
  "id": "cmpl-7fc1be4c-8f5b-4b2f-805f-f8c5086
  "object": "text_completion",
  "created": 1708459650,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.ggu
```

```
    "choices": [
      {
        "text": "Beep boop",
        "index": 0,
        "logprobs": null,
        "finish_reason": "stop"
      }
    ],
    "usage": {
      "prompt_tokens": 198,
      "completion_tokens": 10,
      "total_tokens": 208
    }
  }
```

As a quick validation, make an API request to the /api?message=hi endpoint. If you get a successful response, note that you've correctly completed this sub-task. A decent developer knows what to do once a small feature has been completed.

Subtask 4: MyPy

We have added types to the input and output of our system. FastAPI will validate all inputs and outputs for us, but it won't raise alarms if any section of our code lacks type hints.

The next step is to add a check that ensures all our functions have type hints and that those type hints are correct, without running the server. This can be achieved using static type checkers. In Python, a popular option is mypy.

Mypy can be installed as a Python package, but since this is a development-only package, we will add it with Poetry using the `--group dev` flag, which allows us to separate our development-only tooling from our production-ready libraries:

```
poetry add --group dev mypy
```

If you look at the `pyproject.toml` file, you will see that we now have a new section for developer dependencies, which includes mypy:

```
[dependency-groups]
dev = [
```

```
    "mypy (>=1.18.2,<2.0.0)"
]
```

To run mypy, we can simply use the following command:

```
poetry run mypy -p llm_server
```

If your code is functional, you will probably see the following:

```
Success: no issues found in 2 source files
```

In fact, with all of the Python code you've written for this class, you are very likely to encounter no issues.

While I'd love to say that's it and we're done, this behavior occurs because mypy doesn't report **missing** type annotations by default.

Since we want to learn how to fully typecheck a Python program, we will need to enable strict checking. Run the following:

```
poetry run mypy -p llm_server --strict
```

You should see something like the following:

```
llm_server/main.py:41: error: Function is mis
llm_server/main.py:45: error: Function is mis
Found 2 errors in 1 file (checked 2 source fi
```

As you can see, your file contains several missing type annotation fields. Your task is to fix them and get mypy to pass strict type checking.

In the snippet above, after the `llm_server/main.py` (which is our filename), there is a number for each error. This is a line number, which most tools of this kind use to indicate where you need to fix your code. While it is impossible to determine if your code is structured exactly the same way as ours, we provide a few notes to help in this endeavor, as getting type checking fully working can be exhausting:

> The function for the `/api` endpoint returns a type `Union[CreateCompletionResponse, Iterator[CreateCompletionResponse]]`. You will need to import `Union` and `Iterator` from `typing`

and `CreateCompletionResponse` from `llama-cpp-python`.
Your `healthcheck` function returns a type of `str`.
If you have additional issues due to any extra functions you have created, feel free to add the correct input and output types.

Interim Validation

If you completed this correctly, you should see something like the following when you run the typecheck:

```
Success: no issues found in 2 source files
```

Subtask 5: Async and Await

Now that our application is cleaner and passes our type-check, let's discuss making it faster.

To accomplish this, we shall make use of asynchronous programming.

In your Flask server, Python simply had no idea when to pause or unpause your application. It didn't realize it could pause the LLM inference section, accept a new request, and then resume the LLM inference.

All we want to do is enable our program to do exactly that. Fortunately, in FastAPI, this is remarkably straightforward. All you need to do is use the `async` and `await` keywords.

To make our endpoints asynchronous, we add an `async` keyword before the function definition. For example: `def get_app()` becomes `async def get_app()`.

Doing this gives us the ability to mark the slowest operations in our code with an `await`. This tells FastAPI that while it is waiting for the slowest operation to complete, it can handle other requests, thus processing more connections. The async/await pattern is common across many languages, though their implementations differ.

Make both endpoint functions asynchronous. For the `/api` endpoint, change the LLM invocation line to the following:

```
# import asyncio at the top
resp = await asyncio.to_thread(llm, template,
```

Test both endpoints via curl or a web browser, and they should work correctly.

Subtask 6: Deploying and Testing

Since your goal in this task was to update your `llm_server` and confirm that it is indeed faster with FastAPI, let's deploy it and verify that it works well in AKS.

Let's create a wheel for our server using the following Poetry command:

```
poetry build --clean --format wheel
```

To test that the application works:

1. Modify the Dockerfile to include the new `.whl` package.
2. Build and run the Docker image locally.
3. Make sure you test all endpoints on the running container image locally before pushing it to the ACR.
4. Run the Terraform scripts to provision the infrastructure.
5. Log in to the ACR.
6. Push the Docker image to the ACR.
7. Log in to the AKS cluster.
8. Apply the deployment and service YAML you created in the previous project.
9. Test using the external IP of the service.

Feel free to refer to the Unit 0 project to retrieve the specific commands. While this task is optional, the commands are extremely important and will be reused in every project. It is strongly recommended that you either memorize them or save them somewhere readily accessible.

Submission

```
./submitter -t task1
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform